

# Build with LLMs: Open Source AI for the JVM

Wednesday, September 16 2026, 1-2 PM PST

Silicon Valley Campus, Mountain View, CA, USA



2026

Microsoft

>hackathon\_

SPEAKER

>h\_

# MICROSOFT



**KANNUPRIYA KALRA**



# ***BUILD OPEN. MOVE FAST. SHIP 1.0.***

*The Journey to LLM4S 1.0*



LIVE DEMOS



ARCHITECTURE



LESSONS LEARNED



# Kannupriya Kalra

Co-Founder, LLM4S Foundation • AI thought leader, London

- **Creator of LLM4S:** an AI-native platform for Scala and the JVM
- **Open source:** contributes to the AI and Scala ecosystem through the Scala Center and Google Open Source
- **Google Summer of Code:** led the Scala Center programmes in 2024 and 2025
- **Author:** four AI courses with Manning Publications, New York, with more practical AI learning resources in progress
- **Global speaker:** AI, Scala and open source in more than ten countries
- **Engineering:** built scalable backend systems in functional Scala at Sky, London
- **Mission:** empowering developers through hands-on GenAI projects, open-source contributions and responsible AI education



*With Martin Odersky, creator of the Scala language, at Scala Days 2025 in Lausanne, Switzerland, discussing LLM4S*

# Today's journey

01

## The real AI problem

Why demos work and systems don't

02

## Why Scala and the JVM

Types, errors as values, and the runtime you already run

03

## LLM4S in code

Architecture and real code for every major capability

04

## What we built and learned

Szork, production patterns, wins and failures, the road to 1.0

05

## LLM4S at Microsoft

Scala, the JVM, open source, and what to build this week

# Demos amaze. Systems collapse under real load.

**95%**

of enterprise GenAI pilots  
show no measurable  
business return

MIT NANDA, The GenAI Divide, Jul 2025 (preliminary; 153 leaders, 52 interviews)

**88%**

of AI proofs of concept  
never reach wide  
deployment

Only 4 of every 33 made it to production.

IDC with Lenovo, AI CIO Playbook 2025 (Mar 2025)

## Why pilots stall



### Ungrounded answers

Models hallucinate when nothing ties them to real data.



### Latency and cost at scale

What feels instant in a demo gets slow and expensive under real traffic.

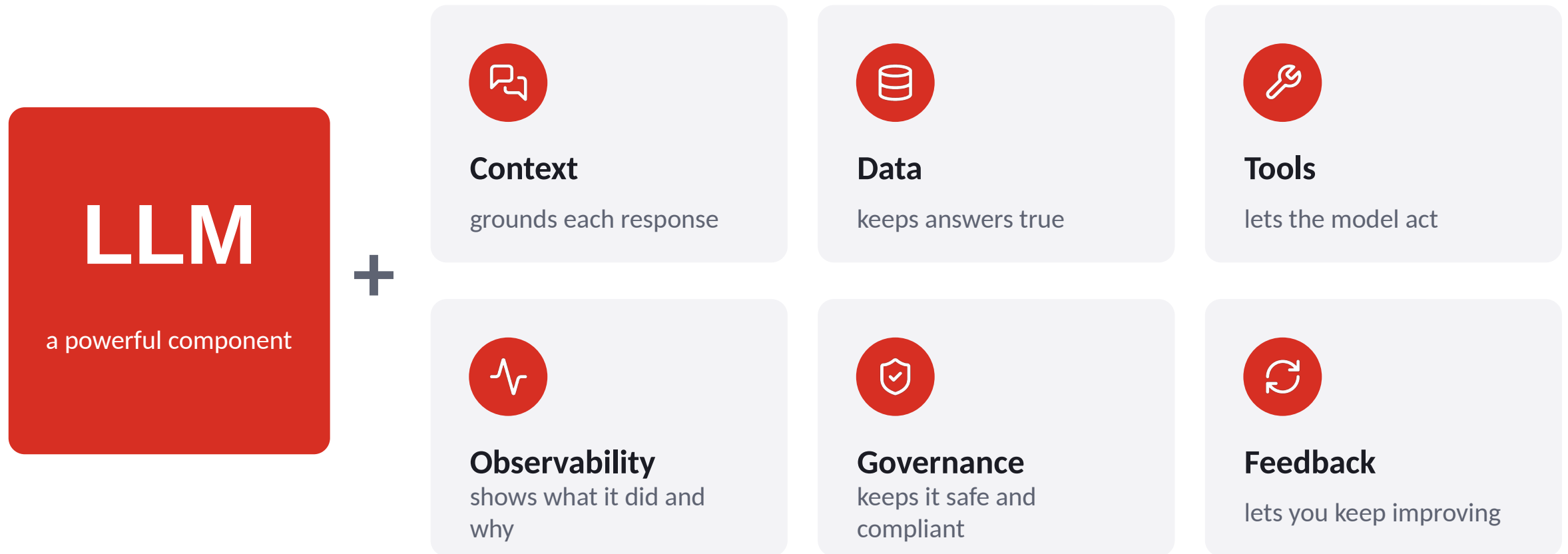


### No way to see or measure

No traces, no metrics, no feedback loop, so no way to improve.

**The shift: from cool prototype to critical system.**

# An LLM is not a product



**LLM4S gives every one of these layers a typed, testable home.**

# From idea to 1.0, in public



# Built by a global community

## Directors and maintainers



### Kannupriya Kalra

Co-founder, LLM4S Foundation. Creator of LLM4S, maintainer and community lead.



### Rory Graves

Director and maintainer, chief architect. Staff engineer and Scala compiler contributor.

## Steering committee

Atul S. Khot

Vitthal Mirji

Iyad M. Issa

Giovanni Ruggiero

Dmitry Mamonov

## Google Summer of Code 2025 contributors



### Elvan Konukseven

Switzerland · Agentic toolkit



### Gopi Trinadh Maddikunta

India · RAG in a box



### Anshuman Awasthi

India · Image, voice and multimodal



### Shubham Vishwakarma

India · Tracing and observability

400+

members

200+

contributors

69

code contributors

1,100+

commits

Including principal and staff engineers from France, Poland and beyond

Sources: [llm4s/llm4s people.md](#) and [contributor graph \(2026\)](#); GSoC 2025, Scala Center

# Real traction, in the last 14 days

GitHub traffic for llm4s/llm4s, the 14 days to 16 Sep 2026

537

git clones

113

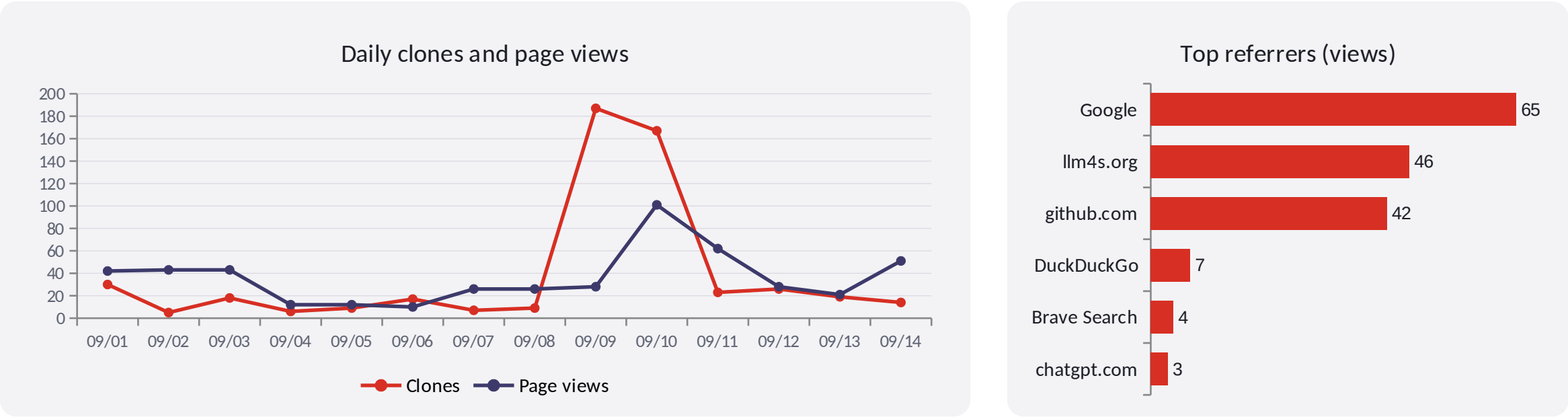
unique cloners

505

page views

206

unique visitors



Source: GitHub repo Insights > Traffic, llm4s/llm4s, captured 16 Sep 2026



# Why Scala for LLM applications?

## Most LLM work happens in Python.

We believe production AI deserves a stronger foundation: one where mistakes surface at compile time, not at 2am.



### Types turn runtime surprises into compile errors

Message roles, config, tool schemas and provider cases are checked before you ship.



### Errors are values

Result[T] makes every failure path explicit and composable.



### Immutable conversations

State you can persist, resume, test and reason about.



### The JVM you already run

Mature libraries, real concurrency, deployed next to your services.

# The typo that compiled and shipped

✗ Before: roles were strings

```
// role is just a String
val msg = Message(
  role    = "asistant", // typo
  content = reply
)

// compiles, passes review,
// misbehaves in production
```

✓ After: roles are an enum

```
// a closed set of cases
enum MessageRole:
  case System, User, Assistant, Tool

MessageRole.Asistant
// error: value Asistant is not
// a member of MessageRole
```

**43** files migrated from strings to types

**6** type classes introduced

**1** compile error instead of a production bug

Simplified from the real LLM4S change (PR #216)

# Errors are values, not surprises

```
val result = for {  
  config    <- Llm4sConfig.provider()  
  client    <- LLMConnect.getClient(config)  
  response <- client.complete(  
    messages = List(UserMessage("Explain MCP")),  
    model     = None  
  )  
} yield response  
  
result match {  
  case Right(c) => println(c.content)  
  case Left(err) => println(s"Error: $err")  
}
```

## 47

try-catch blocks deleted in one safety refactor

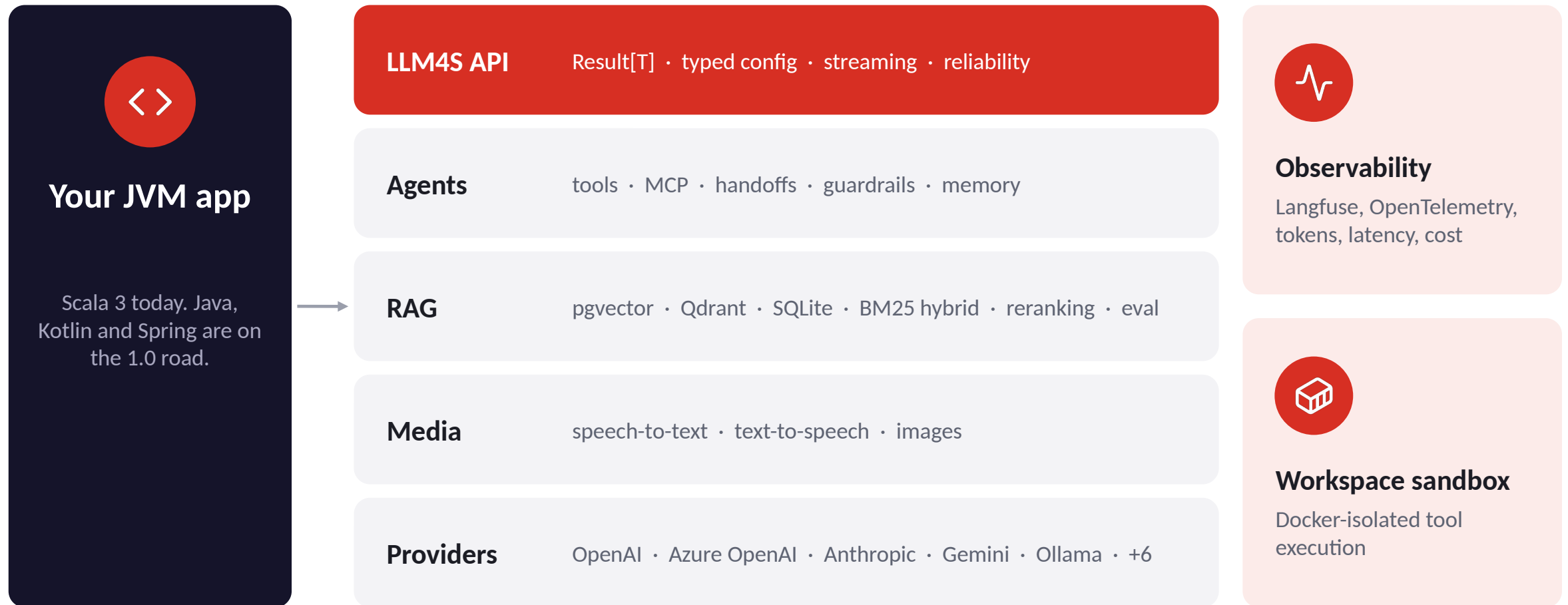
## v0.3.0

removed the last APIs that threw exceptions, in favor of Result

## 1

typed error hierarchy, replacing "Unknown error" for everything

# How LLM4S fits together



# One call, any provider

Every LLM4S call returns a Result. You compose steps, and the compiler makes you handle failure.

```
val result = for {  
  config  <- Llm4sConfig.provider()      // reads env  
  client  <- LLMConnect.getClient(config)  
  response <- client.complete(  
    messages = List(  
      SystemMessage("You are a helpful assistant."),  
      UserMessage("What is Scala?")  
    ),  
    model = None                          // uses config  
  )  
} yield response  
  
result match {  
  case Right(c)  => println(c.content)  
  case Left(err) => println(s"Error: $err")  
}
```

Code and provider list: [llm4s.org docs and README](https://llm4s.org/docs), [llm4s/llm4s v0.4.1](https://llm4s.org/llm4s) (2026)

## Switch model with config, not code

```
export LLM_MODEL=openai/gpt-4o  
# or  
export LLM_MODEL=azure/gpt-4o  
# or, fully local  
export LLM_MODEL=ollama/mistral
```

## Eleven providers, one interface

|          |                     |            |
|----------|---------------------|------------|
| OpenAI   | <b>Azure OpenAI</b> | Anthropic  |
| Gemini   | Ollama              | DeepSeek   |
| Mistral  | Cohere              | OpenRouter |
| Requesty | Z.ai                |            |

# Letting the model act, safely

```
// a plain Scala function
def getWeather(location: String): String =
  s"The weather in $location is sunny, 72F"

val weatherTool = ToolFunction(
  name      = "get_weather",
  description = "Current weather for a location",
  function   = getWeather _
)

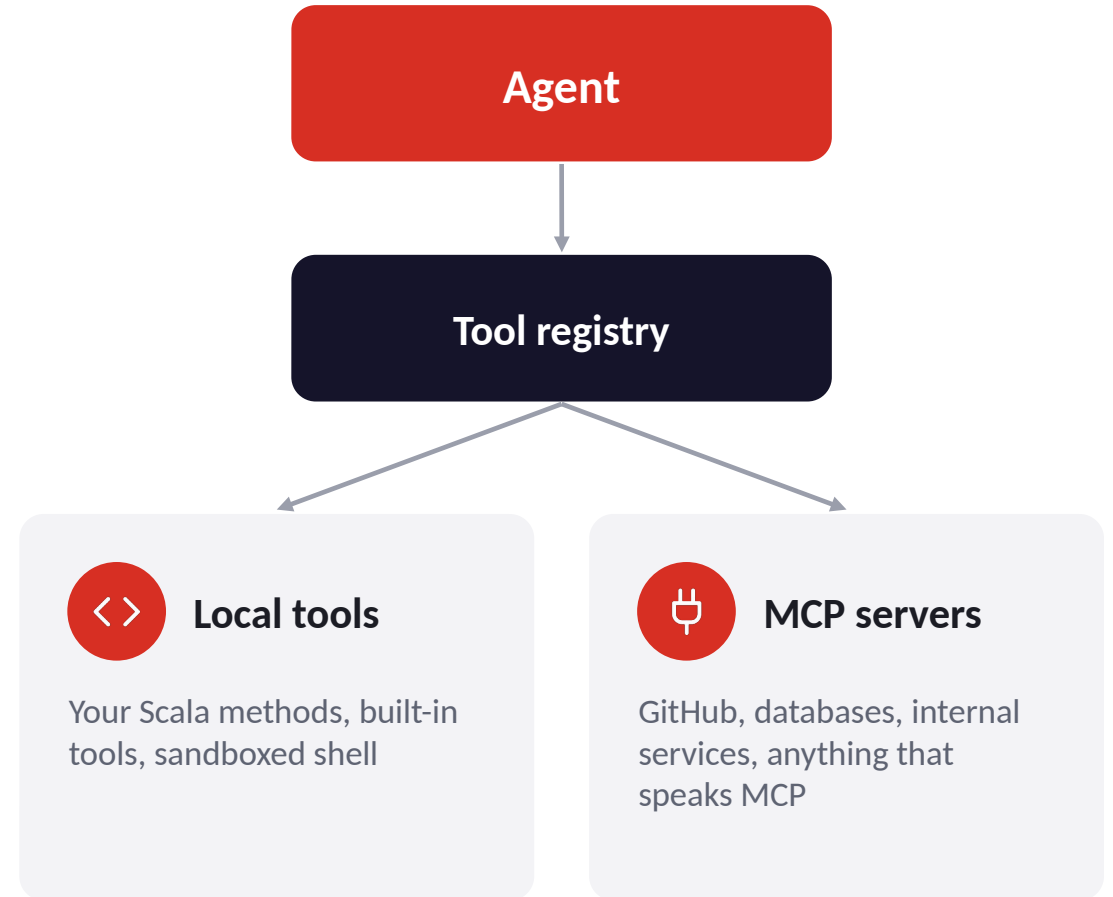
val tools = new ToolRegistry(Seq(weatherTool))
agent.run("What's the weather in Paris?", tools)
```



Built-in tools: date and time, HTTP, web search, files



Tools can run in a Docker-isolated workspace



# Plug in any MCP server

```
val server = MCPServerConfig.streamableHTTP(  
    name      = "mcp-tools-server",  
    url       = "http://localhost:8080/mcp",  
    timeout   = 30.seconds  
)  
  
// local tools and remote MCP tools, one registry  
Using.resource(new MCPToolRegistry(  
    Seq(server), Seq(weatherTool), 10.minutes  
)) { tools =>  
    agent.run(  
        query      = "Convert 100 USD to EUR, then check Paris",  
        tools      = tools,  
        maxSteps   = Some(5)  
    )  
}
```



## One protocol for tools

Connect a server once, and the agent discovers its tools.



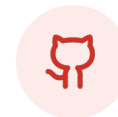
## Mix local and remote

Local tools take precedence when names collide.



## Transport fallback

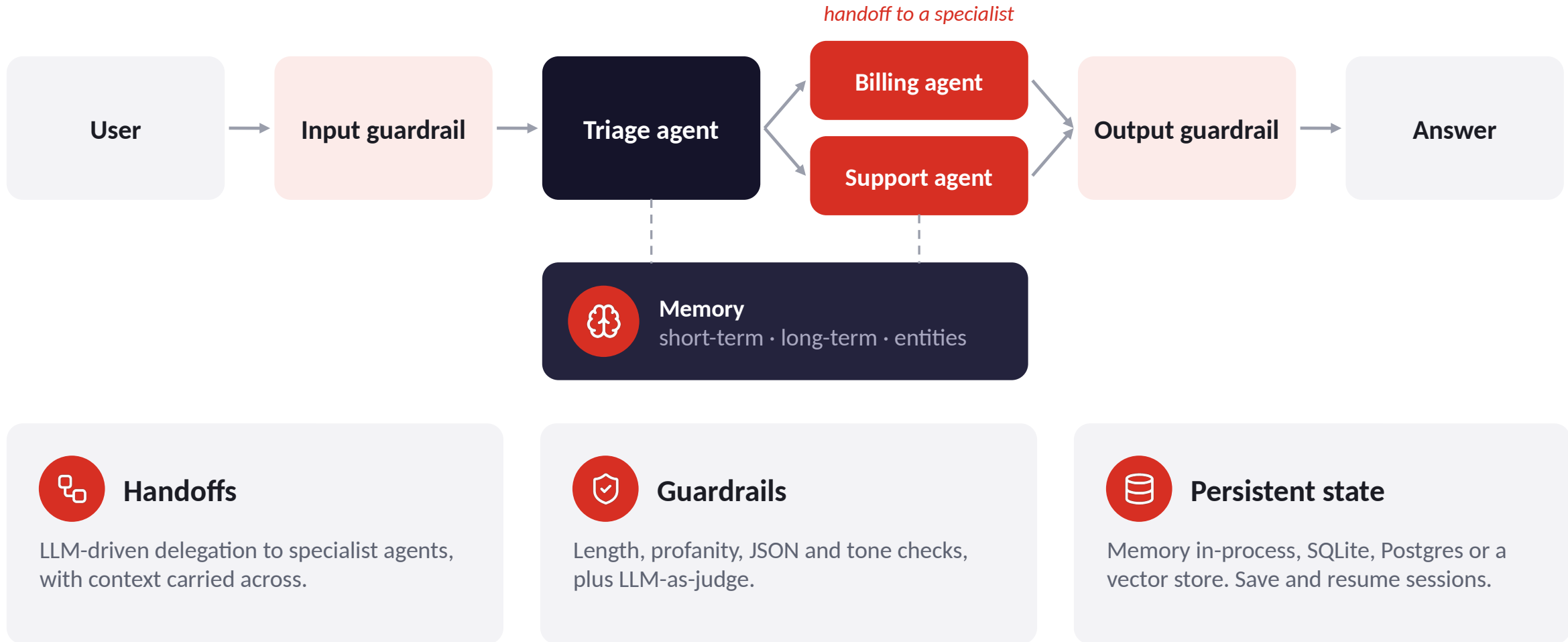
Streamable HTTP, SSE or stdio, handled for you.



## An open standard

Backed across the industry, including Microsoft and GitHub.

# Agents that hand off, check and remember



# Handoffs and guardrails, in a few lines

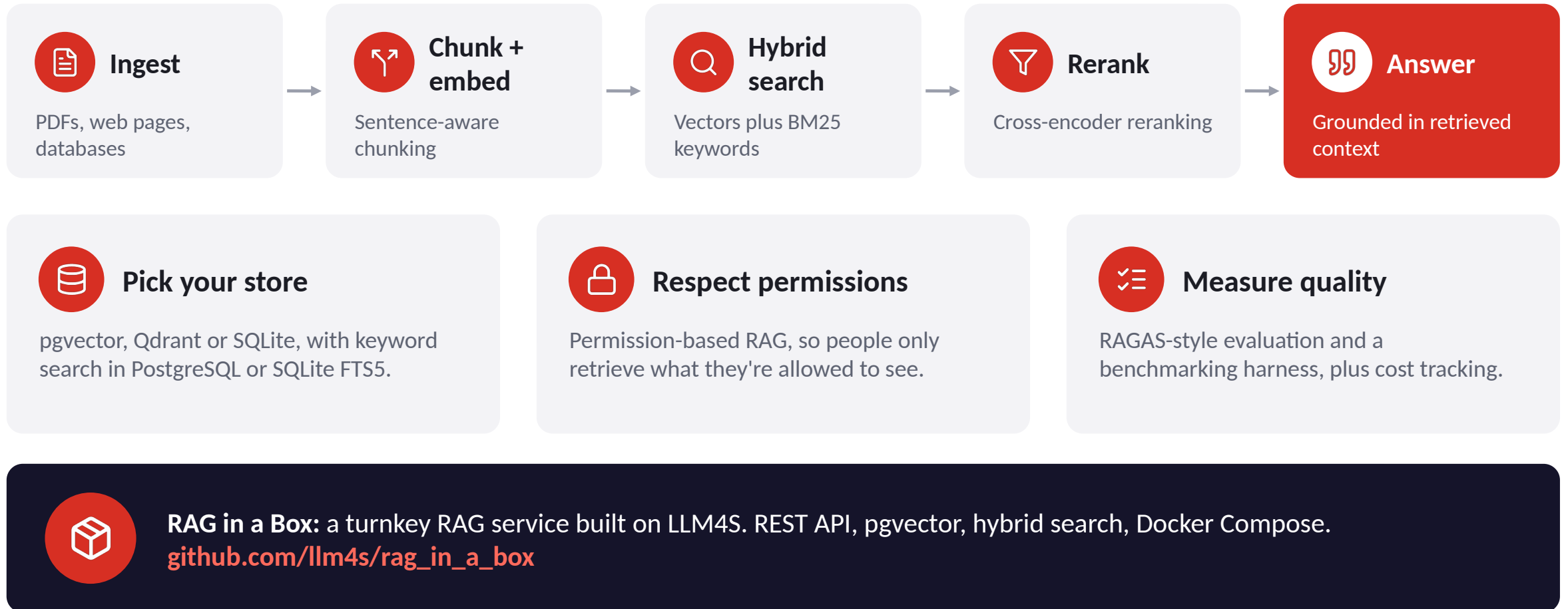
## Hand off to a specialist

```
val mathAgent = new Agent(  
    client      = client,  
    systemMessage = Some(SystemMessage(  
        "You are a math expert."))  
)  
  
mainAgent.run(  
    query      = "What is the integral of x^2?",  
    tools      = ToolRegistry.empty,  
    handoffs   = Seq(  
        Handoff.to(mathAgent, "Math question"))  
)
```

## Guard what goes in and out

```
agent.run(  
    query = userInput,  
    tools = tools,  
    inputGuardrails = Seq(  
        new LengthCheck(min = 1, max = 10000),  
        new ProfanityFilter(),  
        new PromptInjectionDetector()),  
    outputGuardrails = Seq(  
        new JSONValidator(),  
        new PIIMasker())  
)  
// failures return Left(GuardrailError(...))
```

# Grounded by design: retrieve, then generate



# See every step, as it happens

## Stream agent events

```
agent.runWithEvents(query, tools) {  
  case TextDelta(text, _) =>  
    print(text) // real-time tokens  
  
  case ToolCallStarted(_, name, _, _) =>  
    println(s"Calling $name...")  
  
  case ToolCallCompleted(_, name, _, _, ms, _) =>  
    println(s"$name completed in ${ms}ms")  
  
  case AgentCompleted(_, steps, totalMs, _) =>  
    println(s"Done in $steps steps (${totalMs}ms)")  
  
  case _ => ()  
}
```



## Tracing, one setting

```
# Langfuse in production  
TRACING_MODE=langfuse  
LANGFUSE_PUBLIC_KEY=pk-lf-...  
LANGFUSE_SECRET_KEY=sk-lf-...
```

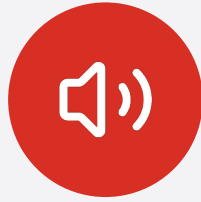
- Console tracing for local development
- OpenTelemetry export module
- Token usage, latency and cost per call
- Raw provider exchange logs for debugging

# Beyond text: voice and images



## Speech-to-text

Turn voice into input for the same agents and tools.



## Text-to-speech

Give your app a voice without leaving the JVM.



## Images

Generate and analyze images through one API.



**Honest labels:** speech and image APIs are marked Experimental for 1.0. They work today, and they may still change. We'd rather tell you now than surprise you later.

# What's already been built



## RAG in a Box

A turnkey RAG service: document ingestion, hybrid search and grounded answers over a REST API.



## Workspace agents

Agents that run commands and work with code inside a Docker-isolated workspace.



## 69 examples

Chat, streaming, agents, guardrails, memory, RAG, tools and reasoning modes. Clone and run.



## Starter kit

`sbt new llm4s/llm4s.g8` creates a working project with CI, formatting and tests.



**Independent visibility:** LLM4S is tracked on Linux Foundation LFX Insights, listed in its LLM API gateways and local inference collection alongside Ollama, vLLM and LiteLLM.

# Szork: an AI Dungeon Master

A voice-controlled text adventure that reimagines Zork. An LLM4S agent runs the game: it narrates the story, manages the world and inventory, and reacts to anything the player says.



## Agent as Dungeon Master

Dynamic story, game state and inventory



## Speak your moves

Speech-to-text for natural commands



## Narrated aloud

Text-to-speech brings the story to life



## Illustrated scenes

AI-generated art for every new location

Live demo, Scala Days 2025

## Watch the recording

EPFL, Lausanne, Switzerland

[youtube.com/watch?v=5od95FbNjil](https://youtube.com/watch?v=5od95FbNjil)



## LLM4S features in play

Agents

Tool calling

Speech-to-text

Text-to-speech

Image generation

Persistent state

[github.com/llm4s/szork](https://github.com/llm4s/szork)

# What worked, what failed, what we learned



## What worked

- Types over strings: whole classes of bugs gone
- Result[T] everywhere instead of exceptions
- A starter kit: setup from 20 minutes to 60 seconds
- Building in public: weekly live coding and GSoC mentoring



## What failed

- One 84k-line core artifact: every user got everything
- Unused JDBC drivers and parsers leaking onto users' classpaths
- "Unknown error" for every failure, early on
- Inconsistent artifact names, and one accidentally mis-published version



## What we learned

- Freeze a small spine, let the edges move
- Module boundaries follow the dependency graph
- Label maturity honestly: Frozen, Beta, Experimental
- Say no to scope: 1.0 targets Scala 3 only

# What 1.0 promises

## Frozen at 1.0

Source and binary compatible across 1.x. Deprecate before removing.

llm4s-core

llm4s-agent

llm4s-observability

OpenAI + compatible clients

Anthropic, Gemini, Ollama

## Beta

Usable now, still hardening. Migration notes ship with every change.

RAG and vector stores

Memory (+ Postgres)

MCP

Shared media types

Community providers

## Experimental

Useful and evolving. Expect the API to change.

Speech

Image generation + processing

Knowledge graph

Today: v0.4.1 is released, and the module split is in progress on main. Binary compatibility is enforced with MiMa checks.

# The road ahead

## On the road to 1.0

- Java facade and Kotlin coroutine API
- Spring Boot starter, Maven and Gradle examples
- Provider capability matrix and contract tests
- Threat model, benchmarks and release gates
- Maintained reference applications



## Also exploring: AI for hardware design

Scala already powers hardware design through Chisel. We're exploring LLM4S agents for that workflow:

- Chisel and FIRRTL integration
- Verification agents that generate tests
- Design space exploration for area, timing and power



### Status: proposal

Hardware roadmap on GitHub. Collaborators welcome.

# Why contribute, and how to join

## Why contribute



### Learn from experienced engineers

Staff engineers and architects review, pair and mentor.



### Shape the design

Your ideas influence where a young framework goes.



### Mentorship programs

GSoC 2025 alumni project, with ideas for GSoC, LFX and European Summer of Code.



### Career impact

Real portfolio work, conference talks and a rare Scala plus AI skill set.

## How to join

### LLM4S Dev Hour

Sundays, 9am London. The community votes on features, then we mob-program, pair and ship.

### Discord

Questions, design discussions and help.

### Using LLM4S at work?

Sponsor the LLM4S Foundation to keep it maintained.



GitHub



Discord

# Built on the same open foundations

LLM4S and Microsoft already meet in open source. Same runtime, same protocols, same home for code.



## OpenJDK

### THE RUNTIME

LLM4S runs on the JVM. Microsoft builds and contributes to OpenJDK.



## MCP

### THE TOOL PROTOCOL

LLM4S agents speak MCP; Microsoft and GitHub joined its Steering Committee (2025).



## GitHub

### WHERE IT'S BUILT

LLM4S is developed in public on GitHub, with MIT-licensed code anyone can use.



## Azure OpenAI

### THE MODELS

A first-class LLM4S provider, on the same code path as every other model.



**The LLM4S side of the umbrella:** the LLM4S Foundation, Google Summer of Code under the Scala Center, a 400+ person community, and independent health tracking on Linux Foundation LFX Insights.

# Scala already runs across Microsoft's world



## Scala inside Microsoft

Microsoft's Java team said its internal systems span Scala, Kotlin and Java (InfoQ, 2021).



## SynapseML

Microsoft's open-source ML library for Spark, published for Scala and pre-installed in Microsoft Fabric.



## Spark on Azure

Azure Databricks and Microsoft Fabric both support Scala for Spark workloads.



## Kafka, from LinkedIn

Apache Kafka was created at LinkedIn (Microsoft since 2016) in Scala and Java.

## Scala in the enterprise

### Apache Spark

The dominant big-data engine, written in Scala. Databricks stewards it and sits on the Scala Center board.

### Akka and Kafka

Widely used JVM infrastructure for concurrency and streaming, built in Scala and Java.

### Financial services and retail

Scala runs data platforms and back-end systems across large enterprises.

**LLM4S brings type-safe agents to where this Scala already runs.**

# Microsoft invests in Java, and runs on it

# 500K+

JVMs in production at Microsoft,  
running hundreds of internal  
systems

*“We use more Java than one can  
imagine.”*

Satya Nadella, 2021 (~5 years ago; figure not updated  
since)

Microsoft for Java Developers blog (2021); ADTmag, InfoQ

## Open-source investment in Java

- 2019** Joined OpenJDK; acquired Java performance firm jClarity
- 2021** Launched Microsoft Build of OpenJDK, a free LTS distribution
- 2021** Strategic member of the Eclipse Foundation and Eclipse Adoptium
- 2021** Joined the Java Community Process, which guides Java's future

## One toolkit for every JVM team

Scala teams use LLM4S today. Artifacts are on Maven Central, so Maven and Gradle builds can pull them in. A Java facade, Kotlin coroutine API and Spring Boot starter are on the 1.0 road, so Java and Kotlin teams get a first-class path too.

# LLM4S in a week

The hackathon runs all week. Here's a realistic arc from empty folder to a deployed AI service.

**MON**

**Set up + first agent**

```
sbt new  
llm4s/llm4s.g8
```

One command scaffolds a working Scala project. Point LLM\_MODEL at Azure OpenAI, run your first agent.

**TUE**

**Give it tools**

```
new  
ToolRegistry(Seq(my  
Tool))
```

Wrap your own functions as tools, or connect an MCP server so the agent can act on real systems.

**WED**

**Ground it in your data**

```
RAG in a Box +  
pgvector
```

Add retrieval over your docs, stored in Azure Database for PostgreSQL. Answers cite real sources.

**THU**

**Make it safe to show**

```
input + output  
guardrails
```

Add length, profanity and prompt-injection checks, PII masking, and turn on tracing to see every step.

**FRI**

**Ship + demo**

```
docker + AKS
```

Package the service in a container, run it on AKS or Container Apps, and demo it with the trace on screen.

# What you could build with LLM4S



## GitHub triage agent

An agent connected to GitHub over MCP that reads new issues, labels them and drafts a first response.

MCP • tools • guardrails



## Internal docs Q&A

Grounded answers over team wikis and runbooks, stored in Azure Database for PostgreSQL with pgvector.

RAG • pgvector • Azure OpenAI



## On-call assistant for JVM services

A triage agent that checks health endpoints and hands off to specialist agents for each system.

Agents • handoffs • tracing



## Guarded support bot

Input and output guardrails, PII masking and full traces, ready for real users.

Guardrails • observability



## A Szork for your team

A voice-driven onboarding adventure that teaches new hires your systems.

Speech • images • agents



## Ship it on Azure

Package your LLM4S service as a container and run it on AKS or Azure Container Apps.

Production patterns

Built on LLM4S capabilities ([llm4s.org](https://llm4s.org)) and Azure services (Microsoft Learn)

## KEY TAKEAWAYS

# Build the cool stuff, and make it work in production.

- 01 Reliable AI is designed in, not tested in at the end.
- 02 Frameworks beat ad-hoc scripts once you have real users.
- 03 Types and Result values make LLM apps predictable.
- 04 LLM4S brings this to the JVM you already run.



Kannupriya Kalra

Engineering Leader | AI | Scala Back-end |  
Google Summer Of Code 2024 & 2025 Org Ad...



Connect on LinkedIn

## Questions?

kannupriya@llm4s.org · rory@llm4s.org  
kannupriyakalra@gmail.com  
llm4s.org · github.com/llm4s

# Thank you!

Star us on GitHub, then come build with us.



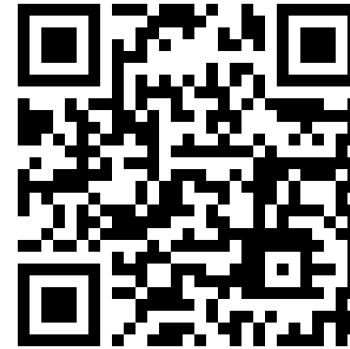
**GitHub**

[github.com/llm4s/llm4s](https://github.com/llm4s/llm4s)



**Docs**

[llm4s.org](https://llm4s.org)



**Discord**

[Join the community](#)